

Archivos de Texto en C++

Lectura y Escritura — Guía Completa

fstream • ifstream • ofstream • Modos • Funciones detalladas

1. Clases para manejo de archivos

C++ usa la biblioteca `<fstream>` para leer y escribir archivos. Tiene tres clases principales:

Clase	Para qué sirve	Encabezado
<code>ifstream</code>	Solo LEER archivos (input file stream)	<code>#include <fstream></code>
<code>ofstream</code>	Solo ESCRIBIR archivos (output file stream)	<code>#include <fstream></code>
<code>fstream</code>	LEER y ESCRIBIR (file stream completo)	<code>#include <fstream></code>

| 💡 Siempre incluir `#include <fstream>` y `#include <string>` al inicio del archivo.

2. Escritura de Archivos — ofstream

2.1 Escritura básica

Crea el archivo si no existe. Si ya existe, lo BORRA y empieza desde cero:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    // Crear/abrir archivo para escritura
    ofstream archivo("datos.txt");

    // Verificar que se abrió correctamente
    if (!archivo.is_open()) {
        cout << "Error: no se pudo abrir el archivo" << endl;
        return 1;
    }

    // Escribir con << igual que cout
    archivo << "Primera línea" << endl;
    archivo << "Segunda línea" << endl;
    archivo << "Número: " << 42 << endl;
    archivo << "Decimal: " << 3.14 << endl;

    // Cerrar el archivo – MUY IMPORTANTE
    archivo.close();

    cout << "Archivo guardado correctamente" << endl;
    return 0;
}
```

2.2 Modos de apertura

El segundo parámetro del constructor define cómo se abre el archivo:

Modo	Constante	Comportamiento
Escritura (por defecto)	ios::out	Borra el contenido existente y escribe desde cero
Agregar al final	ios::app	Agrega al final sin borrar lo existente
Leer	ios::in	Abre para lectura
Binario	ios::binary	Modo binario (no interpreta saltos de línea)
Al inicio	ios::trunc	Borra el archivo al abrir (igual que out)
Crear si no existe	ios::out ios::app	Crea el archivo o agrega si ya existe

```
// Agregar al final del archivo sin borrar lo existente
ofstream archivo("datos.txt", ios::app);
archivo << "Esta línea se agrega al final" << endl;
```

```

archivo.close();

// Combinar modos: leer y escribir a la vez
fstream arch("datos.txt", ios::in | ios::out);

```

2.3 Escribir múltiples tipos de datos

```

#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream arch("registro.txt");

    string nombre = "Carlos";
    int cedula    = 1234567;
    double saldo  = 1500.75;
    bool activo   = true;

    // Escribir variables
    arch << nombre << endl;
    arch << cedula << endl;
    arch << saldo  << endl;
    arch << activo << endl;    // escribe 1 (true) o 0 (false)

    // Formato CSV (separado por comas)
    arch << nombre << "," << cedula << "," << saldo << endl;

    arch.close();
    return 0;
}

```

2.4 Métodos de ofstream

Método	Qué hace	Retorna
open(nombre, modo)	Abre un archivo	void
close()	Cierra el archivo y guarda cambios	void
is_open()	Verifica si el archivo está abierto	bool
operator <<	Escribe datos en el archivo	ofstream&
write(buf, n)	Escribe n bytes en modo binario	ofstream&
flush()	Fuerza escritura del buffer al disco	ofstream&
tellp()	Posición actual del cursor de escritura	streampos
seekp(pos)	Mueve cursor de escritura a posición pos	ofstream&
good()	true si no hay ningún error	bool
fail()	true si hubo un error de operación	bool
bad()	true si hubo error grave (disco lleno,	bool

Método	Qué hace	Retorna
	etc.)	

3. Lectura de Archivos — ifstream

3.1 Leer línea por línea con getline()

getline() lee una línea completa incluyendo espacios. Es la forma más común:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream archivo("datos.txt");

    if (!archivo.is_open()) {
        cout << "Error al abrir el archivo" << endl;
        return 1;
    }

    string linea;

    // Leer línea por línea hasta el final del archivo
    while (getline(archivo, linea)) {
        cout << linea << endl;
    }

    archivo.close();
    return 0;
}

// getline(stream, string) retorna:
// - referencia al stream si leyó correctamente
// - stream con failbit = true si llegó al EOF o hubo error
// Por eso funciona en el while: evalúa a false al terminar el archivo
```

3.2 Leer palabra por palabra con >>

El operador >> lee hasta el primer espacio o salto de línea:

```
ifstream archivo("datos.txt");
string palabra;

// Lee una palabra a la vez (separa por espacios y saltos de línea)
while (archivo >> palabra) {
    cout << palabra << endl;
}

archivo.close();

// Ejemplo: si el archivo tiene 'Hola Mundo'
// Primera iteración: palabra = "Hola"
// Segunda iteración: palabra = "Mundo"
```

3.3 Leer datos de distintos tipos

```
// Si el archivo tiene:
```

```
// Carlos
// 1234567
// 1500.75

ifstream arch("registro.txt");

string nombre;
int cedula;
double saldo;

// Leer cada tipo directamente
getline(arch, nombre); // lee "Carlos"
arch >> cedula;        // lee 1234567
arch >> saldo;          // lee 1500.75

cout << nombre << " - " << cedula << " - " << saldo << endl;

arch.close();
```

3.4 Leer archivo CSV

Separar campos por coma usando getline con delimitador:

```
// Archivo CSV: Carlos,1234567,1500.75
//                      Ana,9876543,2300.00

#include <sstream> // necesario para stringstream

ifstream arch("datos.csv");
string linea;

while (getline(arch, linea)) {
    stringstream ss(linea); // convertir línea en stream
    string nombre, cedStr, saldoStr;

    getline(ss, nombre, ','); // leer hasta la primera coma
    getline(ss, cedStr, ','); // leer hasta la segunda coma
    getline(ss, saldoStr, ','); // leer hasta la tercera coma

    int cedula = stoi(cedStr); // string → int
    double saldo = stod(saldoStr); // string → double

    cout << nombre << " | " << cedula << " | " << saldo << endl;
}

arch.close();
```

3.5 Métodos de ifstream

Método	Qué hace	Retorna
open(nombre)	Abre archivo para lectura	void
close()	Cierra el archivo	void
is_open()	Verifica si está abierto	bool
getline(stream, str)	Lee una línea completa	istream&

Método	Qué hace	Retorna
operator >>	Lee una palabra/valor	istream&
get()	Lee un solo carácter	int (char o EOF)
getline(stream, str, delim)	Lee hasta el delimitador	istream&
eof()	true si llegó al final del archivo	bool
peek()	Lee siguiente char sin avanzar	int
ignore(n, delim)	Salta n caracteres o hasta delim	istream&
tellg()	Posición actual del cursor de lectura	streampos
seekg(pos)	Mueve cursor de lectura a posición	istream&
read(buf, n)	Lee n bytes en modo binario	istream&
good()	true si no hay ningún error	bool
fail()	true si hubo error	bool

4. Lectura y Escritura simultánea — fstream

fstream permite abrir un archivo para leer Y escribir al mismo tiempo:

```
#include <fstream>
using namespace std;

int main() {
    // Abrir para leer y escribir (debe existir el archivo)
    fstream arch("datos.txt", ios::in | ios::out);

    if (!arch.is_open()) {
        cout << "Error al abrir" << endl;
        return 1;
    }

    // Leer el contenido actual
    string linea;
    while (getline(arch, linea)) {
        cout << linea << endl;
    }

    // Limpiar flags de error (eof) antes de escribir
    arch.clear();

    // Mover cursor al final para agregar
    arch.seekp(0, ios::end);
    arch << "Nueva línea agregada" << endl;

    arch.close();
    return 0;
}
```

5. Conversión de tipos al leer

Al leer como string, necesitas convertir a otros tipos:

Función	Convierte	Ejemplo
stoi(str)	string → int	int n = stoi("42")
stol(str)	string → long	long n = stol("123456789")
stof(str)	string → float	float f = stof("3.14")
stod(str)	string → double	double d = stod("3.14159")
stob() (no existe)	—	Usar: bool b = (str == "1")
to_string(n)	número → string	string s = to_string(42)

6. Ejemplo Completo — Registro de Usuarios

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

struct Usuario {
    string nombre;
    int cedula;
    string correo;
};

// Guardar un usuario en el archivo
void guardarUsuario(const Usuario& u) {
    ofstream arch("usuarios.txt", ios::app);
    if (!arch.is_open()) return;
    arch << u.nombre << "," << u.cedula << "," << u.correo << endl;
    arch.close();
}

// Leer todos los usuarios del archivo
vector<Usuario> leerUsuarios() {
    vector<Usuario> lista;
    ifstream arch("usuarios.txt");
    if (!arch.is_open()) return lista;

    string linea;
    while (getline(arch, linea)) {
        stringstream ss(linea);
        string nombre, cedStr, correo;
        getline(ss, nombre, ',');
        getline(ss, cedStr, ',');
        getline(ss, correo, ',');

        Usuario u;
        u.nombre = nombre;
        u.cedula = stoi(cedStr);
        u.correo = correo;
        lista.push_back(u);
    }
    arch.close();
    return lista;
}

int main() {
    // Guardar usuarios
    guardarUsuario({"Carlos", 1234567, "carlos@email.com"});
    guardarUsuario({"Ana", 9876543, "ana@email.com"});

    // Leer y mostrar
    vector<Usuario> usuarios = leerUsuarios();
    for (const auto& u : usuarios) {
        cout << u.nombre << " - " << u.cedula << " - " << u.correo << endl;
    }
    return 0;
}
```

7. Errores comunes y soluciones

Error	Causa	Solución
El archivo no se abre	Ruta incorrecta o no existe	Verificar con <code>is_open()</code> y revisar la ruta
Lee basura o datos incorrectos	Mezclar <code>>></code> y <code>getline()</code>	Después de <code>>></code> , usar <code>arch.ignore()</code> para consumir el salto de línea
No escribe nada	No se cerró el archivo	Llamar <code>arch.close()</code> o usar <code>flush()</code>
Lee solo la primera línea	No está en un <code>while</code>	Poner <code>getline()</code> dentro de <code>while(getline(...))</code>
<code>stoi()</code> lanza excepción	String vacío o no numérico	Verificar que el string no esté vacío antes de convertir
<code>eof()</code> true antes de tiempo	Espacio al final del archivo	Usar <code>while(getline())</code> en lugar de <code>while(!eof())</code>

💡 **NUNCA** usar `while(!arch.eof())` — puede causar que se lea la última línea dos veces. Usar siempre `while(getline(arch, linea))` o `while(arch >> var)`.